

Arquitetura do Projeto — Inspeção PU

Documento para validação técnica e de negócio (inclusive com especialista do mercado de poliuretano). Descreve o problema, a solução, os componentes, as decisões e os limites.

1. Problema que resolve

Conferir, de forma rápida e barata, se peças/rolamentos revestidos de poliuretano estão dentro da medida de fabricação — e documentar isso (laudo/relatório) sem depender de metrologia de bancada para toda peça. Foco em peças até ~200 mm.

2. Princípio de funcionamento

A câmera não mede milímetros sozinha. Precisa de uma referência de escala no quadro (tamanho real conhecido) para converter pixels → mm. A partir daí o app mede diâmetros e compara com um padrão (nominal $\pm$ tolerância). Para rolamento revestido, a referência natural é o furo de aço (preciso e já conhecido pela designação), e a medida de interesse é o diâmetro externo revestido.

3. Componentes

3.1 App de inspeção (núcleo) — roda no celular

- Arquivo único: inspecao-poliuretano/index.html (HTML + CSS + JavaScript, sem build).
- Offline: abre no navegador; pode ser "adicionado à tela inicial".
- Armazenamento local (localStorage): referências, padrões e histórico ficam no aparelho. Exportação em CSV para o PC e relatório em PDF via impressão do navegador.
- Medição: canvas em resolução total da imagem; marcação por toque com lupa de precisão; ferramentas *Distância* (2 pontos) e *Diâmetro* (ajuste de círculo por 3 pontos).
- Calibração de escala: por furo do rolamento ($\varnothing$ interno conhecido) ou por **objeto de referência** (moeda/tampa) cadastrado.
- Detecção automática: OpenCV.js carregado no próprio navegador (HoughCircles). Acha os círculos externo ($\varnothing$ revestido) e interno (furo); com a calibração pelo furo, faz a escala sozinho. Sempre há o modo manual como reserva.

3.2 Catálogo de rolamentos

- 106 rolamentos, séries 6000/6200/6300/6400/6800/6900/16000 (dimensões de contorno ISO 15 / DIN 625: furo d, externo D, largura B).
- Embutido no app (lista JS) para uso offline; e salvo no Supabase (tabela public.rolamentos, RLS com leitura pública) como fonte central/backup.
- Para PU: o app calcula o $\varnothing$ externo revestido esperado = $D + 2 \times$ espessura informada.

3.3 Banco de dados (Supabase)

- Projeto Postgres gerenciado. Tabela public.rolamentos (id, designacao, serie, d_furo, d_externo, largura, revestimento_pu, fonte, created_at).
- Script reproduzível em inspecao-ia/supabase/rolamentos.sql.
- Hoje: o app usa a lista embutida; o Supabase é a cópia central. **Próximo passo possível**: o app ler direto do Supabase (catálogo sempre atualizado e compartilhado entre celulares) via chave pública (anon).

3.4 Trilha opcional de alta precisão (PC/servidor) — inspecao-ia/

- Medição ArUco + OpenCV (Python): gerar_marcador.py, medir_aruco.py, servidor_medida.py. Marcador impresso de tamanho conhecido + homografia imagem→mm (corrige perspectiva) → medida automática (sub-milímetro em boas condições). Exige PC.
- Protótipo de IA de defeitos visuais: treinar.py, servidor.py, embedding.py (detecção de anomalia treinada só com peças boas). Exige PC.

4. Modelo de dados (resumo)

- Referência: { nome, formato (redondo/reto), tamanho_mm }.
- Padrão: { nome, base_D, furo, espessura_PU, tolerância, dims:[{nome, tipo, nominal_mm, tol}], foto? }.
- Inspeção (histórico): { data, peça/lote, padrão, veredito, medidas:[{nominal, medido, ok}], fotos }.
- rolamentos (Supabase): catálogo de dimensões.

5. Decisões de projeto (e por quê)

- App de 1 arquivo, offline, sem servidor: usabilidade no chão de fábrica e zero dependência de TI/infra. Roda em qualquer celular.
- Foco ≤ 200 mm: nesse tamanho, foto + boa referência dá erro baixo ($\sim 1\%$). Em peças grandes a foto perde precisão (cada pixel vale vários mm).
- Calibrar pelo furo: o furo de aço é preciso e conhecido; o PU está no externo (o que se confere). Elimina moeda/papel.
- Ajuste de círculo por 3 pontos: mais estável que 2 toques para peças redondas.
- Auto com fallback manual: produtividade quando a foto é boa, robustez quando não é.
- Catálogo no Supabase: persistência central, backup e base para evoluir (multiusuário, app lendo do banco).
- ArUco/IA como trilha separada: máxima precisão e defeitos visuais, mas exige PC — por isso não é o caminho principal "tudo no celular".

6. Precisão esperada (ordem de grandeza)

- App no celular, peça ≤ 200 mm, foto reta + boa referência: $\sim 1\%$ (ex.: $\pm 0,5$ –1 mm em 50 mm). Depende fortemente de foto e iluminação.
- ArUco + OpenCV (PC): potencial sub-milímetro com setup controlado.
- Para cota crítica apertada, manter paquímetro/micrômetro como referência.

7. Roadmap sugerido

- 1 App lendo o catálogo direto do Supabase (atualização central).
- 2 Espessura do revestimento como critério próprio (reprovar a capa separada do Ø).
- 3 Estação simples (apoio fixo + luz) para repetibilidade e melhor Auto.
- 4 Largura/altura por foto lateral.
- 5 Logo/empresa e assinatura no relatório; marcação do defeito sobre a foto.
- 6 Multiusuário/contas e dashboard de qualidade (PCP).

8. Custos e dependências

- App: zero (HTML estático; OpenCV.js vem de CDN público, cacheado).
- Supabase: plano gratuito atende o catálogo; custo só se virar multiusuário com volume.
- Trilha PC (ArUco/IA): um computador comum; bibliotecas open-source (OpenCV, PyTorch).

9. Riscos e mitigações

- Foto inclinada / iluminação ruim → erro de medida. Mitigação: boas práticas, estação fixa, e a trilha ArUco (corrige perspectiva).
- Auto pode falhar em fundo sem contraste. Mitigação: modo manual sempre disponível.
- Dados só no aparelho → perda ao trocar de celular. Mitigação: exportar CSV; futura leitura/escrita no Supabase.
- Dimensões de catálogo podem variar por fabricante. Mitigação: conferir datasheet; catálogo editável.